

OpenRouter, Groq, and Model API Field Notes

Provider configuration, chat requests, timeouts, rate limits, streaming, errors, and provider wrappers.

Engineering notes and worked references

Contents

1. API keys and environment variables
2. Base URLs and model names
3. Request messages
4. Timeouts
5. Rate limits
6. Streaming
7. Provider response parsing
8. Provider errors
9. Provider abstraction
10. Local debugging

1. API keys and environment variables

Provider API keys should be read from environment variables or an ignored `.env` file during local development.

The code should fail clearly when a required key is missing. It should never ship a real key in the repository.

Example from use

`OPENROUTER_API_KEY` and `GROQ_API_KEY` can be loaded through `python-dotenv` for local work.

Keep an `.env.example` with variable names only.

Things people mix up

- Hard-coding keys in Python.
- Printing the key during startup.
- Committing `.env`.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Provider troubleshooting

When debugging api keys and environment variables, first prove that one small direct provider request works. This separates account, key, model, and network problems from issues in retrieval, sessions, tools, or FastAPI.

After the simple call works, run the same provider client through the application wrapper. Keep the wrapper response consistent so the rest of the project does not need to know every provider-specific field.

Safe diagnostic fields

- Provider and model.
- HTTP status or exception type.
- Provider request ID when available.
- Elapsed time.
- Whether parsing succeeded.

Provider details can change over time, so configuration such as model name should be easy to update. The rest of the application should not depend on a model string being hard-coded in several files.

2. Base URLs and model names

Providers can expose similar chat APIs while using different base URLs and model identifiers. Keep provider-specific values in configuration.

A provider swap should not require editing retrieval, database, or FastAPI code if the chat client is isolated cleanly.

Example from use

Store `base_url` and `model` in environment or settings.

A model name that exists on one provider may not exist on another.

Things people mix up

- Assuming identical model catalogs.
- Scattering base URLs across route functions.
- Calling an unavailable model a parsing problem.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Provider troubleshooting

When debugging base urls and model names, first prove that one small direct provider request works. This separates account, key, model, and network problems from issues in retrieval, sessions, tools, or FastAPI.

After the simple call works, run the same provider client through the application wrapper. Keep the wrapper response consistent so the rest of the project does not need to know every provider-specific field.

Safe diagnostic fields

- Provider and model.
- HTTP status or exception type.
- Provider request ID when available.
- Elapsed time.
- Whether parsing succeeded.

Provider details can change over time, so configuration such as model name should be easy to update. The rest of the application should not depend on a model string being hard-coded in several files.

3. Request messages

Both providers can be used with chat-style messages. Build the messages in one place so system instructions, history, retrieved context, and user input are easy to inspect.

The provider client should receive already prepared application context rather than deciding which session history belongs to the user.

Example from use

`build_messages(session, retrieved_chunks, user_message)` can return the final list.

Log message count and token estimates, not secrets.

Things people mix up

- Loading database history inside the low-level HTTP function.
- Sending raw documents and retrieval context twice.
- Changing role order between providers.

Quick check

- Can the idea be explained without relying on one memorized definition?

- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Provider troubleshooting

When debugging request messages, first prove that one small direct provider request works. This separates account, key, model, and network problems from issues in retrieval, sessions, tools, or FastAPI.

After the simple call works, run the same provider client through the application wrapper. Keep the wrapper response consistent so the rest of the project does not need to know every provider-specific field.

Safe diagnostic fields

- Provider and model.
- HTTP status or exception type.
- Provider request ID when available.
- Elapsed time.
- Whether parsing succeeded.

Provider details can change over time, so configuration such as model name should be easy to update. The rest of the application should not depend on a model string being hard-coded in several files.

4. Timeouts

Network calls need a timeout so a provider problem does not leave the request hanging forever.

Choose a timeout that fits the model and expected response length, then handle the timeout as a normal error path.

Example from use

A timeout can return a controlled 'provider did not respond in time' message.

One limited retry may be appropriate for transient timeouts.

Things people mix up

- No timeout.
- A 5-minute timeout for a small chat turn.
- Retrying forever.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Provider troubleshooting

When debugging timeouts, first prove that one small direct provider request works. This separates account, key, model, and network problems from issues in retrieval, sessions, tools, or FastAPI.

After the simple call works, run the same provider client through the application wrapper. Keep the wrapper response consistent so the rest of the project does not need to know every provider-specific field.

Safe diagnostic fields

- Provider and model.
- HTTP status or exception type.
- Provider request ID when available.
- Elapsed time.
- Whether parsing succeeded.

Provider details can change over time, so configuration such as model name should be easy to update. The rest of the application should not depend on a model string being hard-coded in several files.

5. Rate limits

Providers may limit request rate or token usage. A rate-limit response should be distinguished from invalid authentication or malformed input.

For a small project, a simple backoff or user-facing retry message is usually enough.

Example from use

Log the status and provider request ID.

Avoid a tight immediate retry loop.

Things people mix up

- Treating rate limit as a model hallucination.
- Retrying ten times instantly.
- Changing prompts when the problem is account quota.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Provider troubleshooting

When debugging rate limits, first prove that one small direct provider request works. This separates account, key, model, and network problems from issues in retrieval, sessions, tools, or FastAPI.

After the simple call works, run the same provider client through the application wrapper. Keep the wrapper response consistent so the rest of the project does not need to know every provider-specific field.

Safe diagnostic fields

- Provider and model.
- HTTP status or exception type.
- Provider request ID when available.
- Elapsed time.

- Whether parsing succeeded.

Provider details can change over time, so configuration such as model name should be easy to update. The rest of the application should not depend on a model string being hard-coded in several files.

6. Streaming

Streaming sends partial generated text as it becomes available. Non-streaming waits for the full response.

Streaming improves perceived latency for long text but makes structured output and tool handling more complicated. A capstone does not need streaming unless it serves a real purpose.

Example from use

Use non-streaming for simple JSON parsing.

If streaming, collect the final structured content carefully.

Things people mix up

- Adding streaming only because it looks advanced.
- Trying to `json.loads` each partial token.
- Mixing streaming and non-streaming response formats in one route without a clear contract.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Provider troubleshooting

When debugging streaming, first prove that one small direct provider request works. This separates account, key, model, and network problems from issues in retrieval, sessions, tools, or FastAPI.

After the simple call works, run the same provider client through the application wrapper. Keep the wrapper response consistent so the rest of the project does not need to know every provider-specific field.

Safe diagnostic fields

- Provider and model.
- HTTP status or exception type.
- Provider request ID when available.
- Elapsed time.
- Whether parsing succeeded.

Provider details can change over time, so configuration such as model name should be easy to update. The rest of the application should not depend on a model string being hard-coded in several files.

7. Provider response parsing

The provider response contains metadata plus the generated assistant content. Client libraries often simplify access, but application code should still handle missing or unexpected fields.

Keep parsing in the provider client layer so the rest of the app receives a consistent internal response.

Example from use

Return text, provider request ID, model, and usage metadata from one wrapper.

Raise a controlled parsing error when the content field is absent.

Things people mix up

- Accessing deeply nested dictionaries in every route.
- Assuming every provider returns identical usage fields.
- Returning raw provider response objects to the API client.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Provider troubleshooting

When debugging provider response parsing, first prove that one small direct provider request works. This separates account, key, model, and network problems from issues in retrieval, sessions, tools, or FastAPI.

After the simple call works, run the same provider client through the application wrapper. Keep the wrapper response consistent so the rest of the project does not need to know every provider-specific field.

Safe diagnostic fields

- Provider and model.
- HTTP status or exception type.
- Provider request ID when available.
- Elapsed time.
- Whether parsing succeeded.

Provider details can change over time, so configuration such as model name should be easy to update. The rest of the application should not depend on a model string being hard-coded in several files.

8. Provider errors

Authentication errors, unavailable models, rate limits, timeouts, and server errors should have distinct log messages.

The user-facing message can remain simple while the server log records the provider and status.

Example from use

401/403 usually indicate credentials or account permission.

429 indicates rate or quota pressure.

5xx can indicate temporary provider problems.

Things people mix up

- Showing the full provider error body if it contains sensitive request content.
- Calling every non-200 error 'timeout'.

- Suppressing status codes.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Provider troubleshooting

When debugging provider errors, first prove that one small direct provider request works. This separates account, key, model, and network problems from issues in retrieval, sessions, tools, or FastAPI.

After the simple call works, run the same provider client through the application wrapper. Keep the wrapper response consistent so the rest of the project does not need to know every provider-specific field.

Safe diagnostic fields

- Provider and model.
- HTTP status or exception type.
- Provider request ID when available.
- Elapsed time.
- Whether parsing succeeded.

Provider details can change over time, so configuration such as model name should be easy to update. The rest of the application should not depend on a model string being hard-coded in several files.

9. Provider abstraction

A small wrapper function can make OpenRouter and Groq look similar to the rest of the application.

Do not overbuild a large framework. A function or small class that accepts messages and returns text is enough for a capstone.

Example from use

`chat(messages, provider='groq')` -> internal response object.

Keep temperature and model config outside FastAPI routes.

Things people mix up

- Building a plugin system for two providers.
- Leaking provider-specific fields into every application function.
- Making provider swapping a requirement when the project only needs one working provider.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the

bug less mysterious.

Provider troubleshooting

When debugging provider abstraction, first prove that one small direct provider request works. This separates account, key, model, and network problems from issues in retrieval, sessions, tools, or FastAPI.

After the simple call works, run the same provider client through the application wrapper. Keep the wrapper response consistent so the rest of the project does not need to know every provider-specific field.

Safe diagnostic fields

- Provider and model.
- HTTP status or exception type.
- Provider request ID when available.
- Elapsed time.
- Whether parsing succeeded.

Provider details can change over time, so configuration such as model name should be easy to update. The rest of the application should not depend on a model string being hard-coded in several files.

10. Local debugging

When a provider call fails, print or log the safe request shape: model, message count, content lengths, timeout, and status.

A reproducible curl or small Python request is useful for separating provider problems from the rest of the application.

Example from use

Test one short 'hello' request before debugging the whole RAG pipeline.

Use the same environment variables the app uses.

Things people mix up

- Debugging provider auth through the full FastAPI/RAG stack first.
- Using a different key or model in the test than the app.
- Copying secrets into screenshots or issue descriptions.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Provider troubleshooting

When debugging local debugging, first prove that one small direct provider request works. This separates account, key, model, and network problems from issues in retrieval, sessions, tools, or FastAPI.

After the simple call works, run the same provider client through the application wrapper. Keep the wrapper response consistent so the rest of the project does not need to know every provider-specific field.

Safe diagnostic fields

- Provider and model.

- HTTP status or exception type.
- Provider request ID when available.
- Elapsed time.
- Whether parsing succeeded.

Provider details can change over time, so configuration such as model name should be easy to update. The rest of the application should not depend on a model string being hard-coded in several files.